Isabella Hernandez Restrepo y
Miguel Angel Rodriguez Uran

Método de Gauss-Seidel y Jacobi

Camilo Betancur Arango y
Maria Paula Cruz Rios



Él es Cricg.

Cricg sabe de análisis numérico.

¡Hola Cricg!

Contenidos

- Introducción
- Desarrollo matemático
- Algoritmo
- Implementación en Python
- Aplicación en física

Introducción

Jacobi

Resuelve sistemas de ecuaciones lineales Ax = b de forma iterativa, sin invertir la matriz.

Es el método iterativo más simple y es la base teórica de la que se derivó Gauss-Seidel.

Consiste en despejar cada incógnita de su respectiva ecuación y partir de una aproximación inicial $x^{(0)}$, y calcular aproximaciones sucesivas usando los valores de la iteración anterior sin actualizar los datos a mitad del proceso.

Introducción

Gauss - Seidel

Igual que Jacobi, resuelve sistemas de ecuaciones lineales Ax = b de forma iterativa, sin necesidad de invertir la matriz ni hacer eliminación completa. La idea es la misma que Jacobi, solo que a partir de la aproximación inicial $x^{(0)}$, genera una secuencia de aproximaciones $x^{(1)}$, $x^{(2)}$, ... que converjan a la solución real, usando en cada ecuación los valores más recientes disponibles (a diferencia de Jacobi, que usa solo los valores de la iteración anterior).

La importancia de ambos métodos radica en su simplicidad conceptual, su utilidad cuando A es grande y dispersa, su capacidad para ser paralelizado en computadoras modernas y en servir como base teórica para comprender métodos iterativos más avanzados

```
Función Jacobi(A, b, x_inicial, tolerancia, max_iteraciones):
    x_viejo = x_inicial

    Para k desde 1 hasta max_iteraciones:
        x_nuevo = nuevo_vector(tamaño_n)

        Para i desde 1 hasta n:  // Recorre cada ecuación (fila i)
            suma = 0

            Para j desde 1 hasta n:  // Multiplica coeficientes por los valores anteriores
                Si j ≠ i entonces:
                    suma = suma + A[i][j] * x_viejo[j]
                Fin Si
            Fin Para

            // Despeja x[i] usando los valores de la iteración anterior
            x_nuevo[i] = (b[i] - suma) / A[i][i]
        Fin Para

        Si distancia(x_nuevo, x_viejo) < tolerancia entonces:
            Retornar x_nuevo  // El método ya convergió
        Fin Si

        x_viejo = copia(x_nuevo)  // Prepara los valores para el siguiente ciclo
    Fin Para

    Retornar x_nuevo
```

Pseudo-código del algóritmo del método de Jacobi

Desarrollo matemático

Jacobi

El método funciona bajo 2 suposiciones:

- **1.** El sistema tiene solución única.
- **2.** La matriz de coeficientes no tiene ceros en su diagonal principal.

Deducción: Sea el sistema:

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + & \cdots & +a_{1n}x_n & = & b_1 \\ \vdots & & & \ddots & \\ a_{n1}x_1 + a_{n2}x_2 + & \cdots & +a_{nn}x_n & = & b_n \end{cases}$$

O también visto en su forma matricial, $Ax = b$, con A siendo la matriz de coeficientes, $x = \begin{pmatrix} x_1 \\ \cdots \\ x_n \end{pmatrix}$ el vector de incógnitas ya definido, y b es el vector de términos independientes.



Paso 1.

Despejar cada una de las incógnitas $x_1, x_2, \cdots, x_n$. Así,

$$x_1 = \frac{1}{a_{11}}(b1 - a_{12}x_2 - a_{13}x_3 - \cdots - a_{n1}x_n)$$

e igualmente para cada valor de x. En general,

$$x_n = \frac{1}{a_{nn}}(bn - a_{n1}x_2 - a_{n2}x_3 - \cdots - a_{n,n-1}x_{n-1})$$

También podemos hallar las x de forma matricial. Descompongamos a A en dos matrices; D es una matriz diagonal con la diagonal principal de los elementos de A, R contiene ceros en la diagonal y las entradas faltantes de A en el resto de sus entradas, de forma ordenada.

De este modo, el sistema queda:

$$(D + R)x = b \rightarrow Dx + Rx = b$$

Despejando a x, nos queda

$$x = D^{-1}(b - Rx)$$

Esta ecuación no aporta una solución por sí misma, pero podemos utilizarla de forma recursiva de modo que utilicemos los valores hallados de cierta suposición inicial de x^k para hallar un nuevo vector de soluciones x^{k+1} (una aproximación posterior a la primera). Así,

$$x^{k+1} = D^{-1}(b - Rx^k)$$

Paso 2.
El método de Jacobi propone que el vector inicial (x^0 sea igual a 0.
$x_1 = x_2 = \cdots = x_n = 0$

Paso 3.
Con esta suposición, reemplazamos cada uno de los valores de x en cada una de las ecuaciones en donde despejamos las n incógnitas.

De este modo,

$$x_1 = \frac{b_1}{a_{11}}, x_2 = \frac{b_2}{a_{22}}, \cdots\cdots, x_n = \frac{b_n}{a_{nn}}$$

Y en general:

$$x_n^{(k+1)} = \frac{b_n - (a_{n1}x^{(k_2)} + a_{n2}x^{(k_3)} + \cdots + a_{nn-1}x_{n-1}^{(k)})}{a_{nn}} \quad (1)$$

Para mayor claridad, esto quiere decir que reemplazamos $x^{(0)}$ en la ecuación (1), en donde obtenemos un nuevo vector, $x^{(1)}$. Así, repetimos el proceso hasta que la norma entre dos vectores consecutivos k y $k+1$ sea menor a una tolerancia preestablecida, definida como:

$$\theta = \sqrt{(x_1^{(k+1)} - x_1^{(k)})^2 + (x_2^{(k+1)} - x_2^{(k)})^2 + \cdots + (x_n^{(k+1)} - x_n^{(k)})^2}$$

Convergencia visual

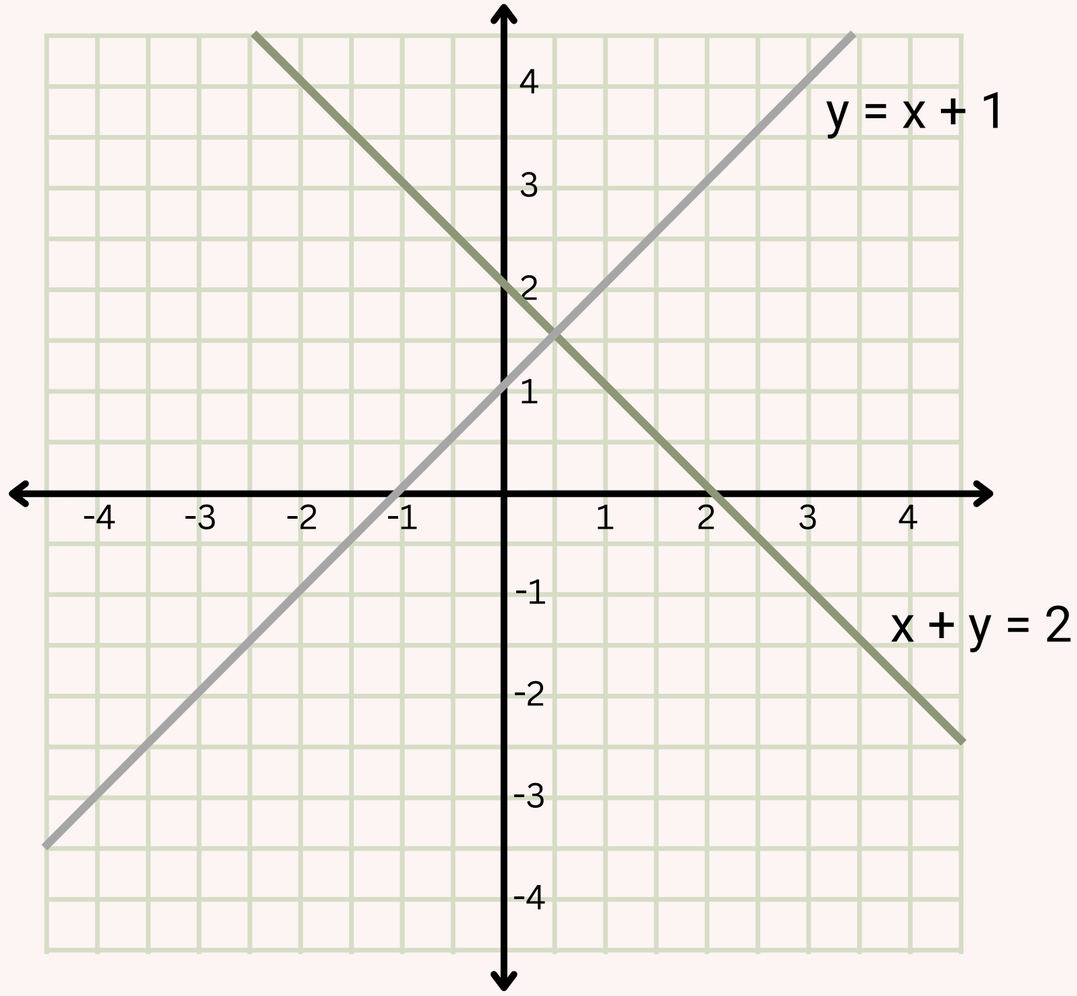
Jacobi

Se tiene el sistema de ecuaciones:

$$x + y = 2$$
$$y = x + 1$$

La solución es x = 0.5, y = 1.5

Las soluciones de dos ecuaciones simultáneas corresponden a la intersección de sus gráficas.



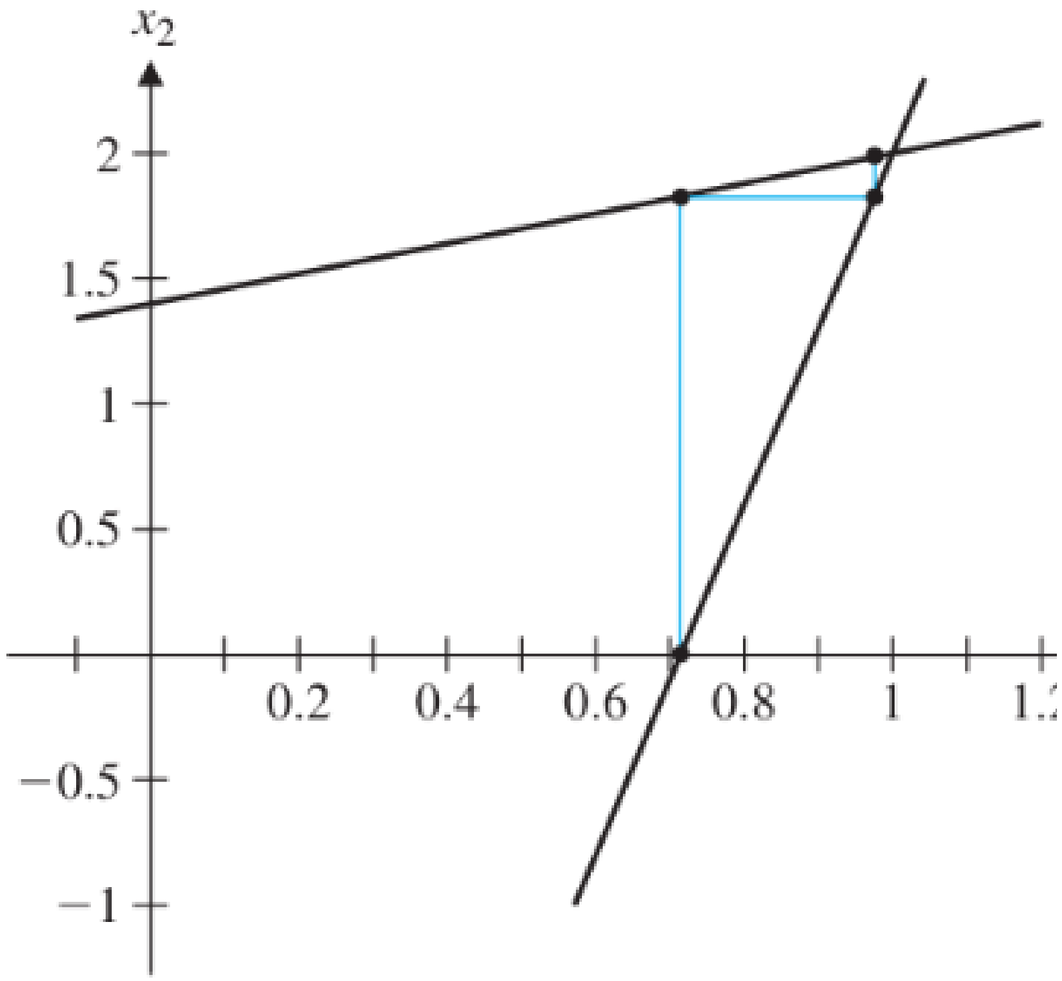


Figura 2.27
Iterados convergentes

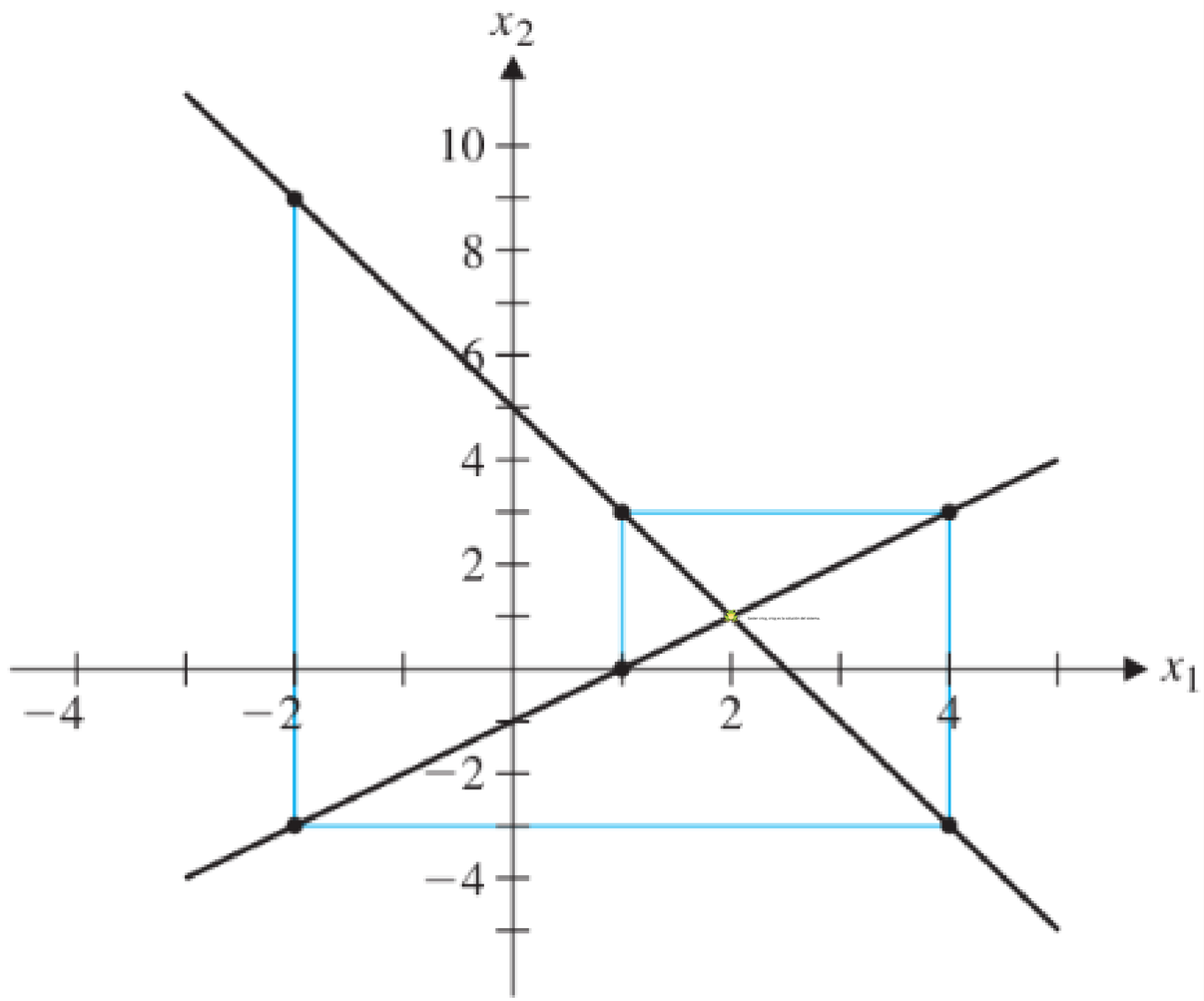


Figura 2.28
Iterados divergentes

Tomado de: Poole, David Álgebra lineal, Una introducción moderna Tercera Edición

Desarrollo matemático

Gauss - Seidel

Deducción: Para la i-ésima ecuación del sistema, aislamos x_i (como en el método anterior:

$$a_{ii}x_i = b_i - \sum_{j=1}^{i-1} a_{ij}x_j - \sum_{j=i+1}^{n} a_{ij}x_j$$

Las incógnitas con índice $j < i$ toman el valor recién actualizado de la iteración $(k+1)$, mientras que las incógnitas con $j > i$ conservan la aproximación de la iteración anterior (k):

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^{n} a_{ij}x_j^{(k)} \right), \quad i = 1, 2, \ldots, n \qquad (2)$$

Desplegado para las primeras ecuaciones, el sistema se sustituye progresivamente:

$$x_1^{(k+1)} = \frac{1}{a_{11}} \left(b_1 - a_{12}x_2^{(k)} - a_{13}x_3^{(k)} - \cdots - a_{1n}x_n^{(k)} \right)$$

$$x_2^{(k+1)} = \frac{1}{a_{22}} \left(b_2 - a_{21}x_1^{(k+1)} - a_{23}x_3^{(k)} - \cdots - a_{2n}x_n^{(k)} \right)$$



Forma matricial: La matriz A se descompone en $A = L + D + U$, donde:

- D: Matriz diagonal conteniendo los elementos de la diagonal de A.
- L: Matriz triangular inferior estricta.
- U: Matriz triangular superior estricta.

Sustituyendo esta descomposición en la ecuación lineal $Ax = b$:

$$(L + D + U)x = b \implies (D + L)x + Ux = b$$

Despejando el término x correspondiente al estado futuro $(k + 1)$ en el lado izquierdo:

$$(D + L)x^{(k+1)} = b - Ux^{(k)}$$

Multiplicando por la inversa $(D + L)^{-1}$ (la cual siempre existe si $a_{ii} \neq 0$), obtenemos la forma matricial de Gauss-Seidel:

$$x^{(k+1)} = (D + L)^{-1} \left(b - Ux^{(k)} \right) \tag{3}$$

Note que en el método de Jacobi la forma matricial equivale a $x^{(k+1)} = D^{-1} \left(b - (L + U)x^{(k)} \right)$, puesto que no se reutilizan las variables actualizadas dentro del mismo paso.

Criterios d
converger



tricta: $|a_{ii}| > \Sigma_{j \neq i}$

en este caso

no suficientes.

tricta: $|a_{ii}| > \Sigma_{j \neq i}$

mbién garantiza

Criterios de convergencia

Ambos métodos comparten los condiciones de convergencia suficientes básicas. Para garantizar ambos converjan hacia la solución exacta x, se exige por lo general que la matriz de coeficientes A sea estrictamente diagonal dominante por filas, la cual posee dos condiciones:

- Necesaria:$|a_{ii}| > |a_{ij}|$.
- Suficiente: $|a_{ii}| > \sum_{j \neq i}^{n} |a_{ij}|, \quad \forall i \in \{1, 2, \ldots, n\}$
- Necesaria y suficiente: Los métodos convergen sii el radio espectral de la matriz de la iteración j ($\rho(A_j)$, es menor que 1.

Hay otro criterio que garantiza la convergencia SÓLO para **Gauss Seidel**: Si la matriz de coeficientes A es simétrica y definida positiva, el método siempre converge, sin importar el valor inicial.

Ventajas y limitaciones

Jacobi

- Muy simple de implementar y entender.
- Totalmente paralelizable: cada x_i se calcula independientemente de las demás en la misma iteración.
- Requiere solo la fila i de A para calcular x_i.
- Evita la acumulación de errores de precisión que sufren los métodos directos al realizar miles de divisiones sucesivas en sistemas de gran escala.
- Converge más lento que Gauss-Seidel (no aprovecha info actualizada).
- Requiere el doble de memoria (vector actual + vector nuevo) frente a Gauss-Seidel.
- No converge para cualquier sistema.



Ventajas y limitaciones

Gauss - Seidel

- Converge más rápido que Jacobi (usa info actualizada de inmediato)
- Bajo costo de memoria (se puede sobrescribir el vector solución)
- Control del error.
- Eficiente para sistemas grandes y dispersos.

- No converge para cualquier sistema (depende de las condiciones anteriores).
- Es inherentemente secuencial (a diferencia de Jacobi, que es paralelizable), porque cada x_i depende de los valores recién calculados.
- Sensible a la elección del valor inicial en cuanto a velocidad de convergencia (aunque no afecta si converge o no, si se cumplen las condiciones).

Algoritmo Jacobi

Algoritmo Método de Jacobi

función Jacobi (A, x^0)

//x^0 es una aproximación inicial a la solución//

para $k \leftarrow 1$ **hasta** convergencia **hacer**

para $i \leftarrow 1$ **hasta** n **hacer**

$\sigma \leftarrow 0$

para $j \leftarrow 1$ **hasta** n **hacer**

si $j \neq i$ **entonces**

$$\sigma = \sigma + a_{ij} x_j^{(k-1)}$$

fin para

$$x_i^{(k)} = \frac{(b_i - \sigma)}{a_{ii}}$$

fin para

comprobar si se alcanza convergencia

fin para

Algoritmo Gauss - Seidel

Algoritmo Método de Gauss-Seidel

función Gauss-Seidel (A, x^0)

//x^0 es una aproximación inicial a la solución//

para $k \leftarrow 1$ **hasta** convergencia **hacer**

para $i \leftarrow 1$ **hasta** n **hacer**

$\sigma \leftarrow 0$

para $j \leftarrow 1$ **hasta** n **hacer**

si $j \neq i$ **entonces**

$$\sigma = \sigma + a_{ij} x_j$$

fin para

$$x_i = \frac{(b_i - \sigma)}{a_{ii}}$$

fin para

comprobar si se alcanza convergencia

fin para

Implementación en python

Aplicación en física

Distribución de temperatura en estado estacionario:

- Fenómeno físico: conducción de calor en una placa con bordes a temperatura fija, hasta alcanzar equilibrio térmico.

- Ecuación gobernante: Laplace,

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = 0$$

(ley de Fourier, sin fuentes internas).

- Discretización (diferencias finitas):

$$T_{i,j} = \frac{T_{i+1,j} + T_{i-1,j} + T_{i,j+1} + T_{i,j-1}}{4}$$

cada nodo es el promedio de sus 4 vecinos.

- ¿Por qué Gauss-Seidel? El sistema es grande y disperso (matriz diagonalmente dominante → convergencia garantizada); al usar de inmediato los valores actualizados, el calor "se propaga" más rápido por la malla en cada barrido → converge en menos iteraciones.

Aplicación en física

Potencial eléctrico en electrostática

- Fenómeno físico: potencial V entre electrodos/placas sin carga libre en la región interior.
- Ecuación gobernante: Laplace: $\nabla^2 V = 0$

  Ley de Gauss; con carga, es Poisson $\nabla^2 V = -\rho/\varepsilon_0$
- Discretización (diferencias finitas):

$$V_{i,j} = \frac{V_{i+1,j} + V_{i-1,j} + V_{i,j+1} + V_{i,j-1}}{4}$$

  Mismo tipo de sistema disperso que en el caso térmico.

- ¿Por qué Jacobi? Cada nodo se actualiza solo con valores de la iteración anterior, osea que es un cálculo totalmente independiente entre nodos; ideal para paralelizar en GPU en simulaciones de campos electromagnéticos con millones de nodos (lentes electrostáticas, aceleradores de partículas). Converge más lento que Gauss-Seidel, pero cada iteración es mucho más rápida en hardware paralelo.

Cricg lo sabe todo.

